شرح سطر بسطر ٢٧ — خزانة رِزمة (24): العناصر والشراء والرسم والشاشة

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

يزيّن الطالب شخصيته بنجومه: إكسسوارات (قبعة تخرّج، شال، نظارة، نجمة إتقان) وألوان (أخضر، برتقالي، سماوي، بنفسجي، عسلي). النجوم تفتح الزينة **فقط**؛ أما الدروس فمفتوحة دائمًا (وهذا مكتوب في الشاشة).

الرصيد = نجوم الطالب − ما أنفقه، يُحسب ولا يُخزَّن. الشراء في معاملة قاعدة بيانات واحدة: فحوص (مملوك؟ مقفل؟ يكفي الرصيد؟) ثم الكتابة؛ وجدول المشتريات مفتاحه (طالب، عنصر) فلا يُخصم السعر مرتين حتى لو تزامن نقران.

المجاني (السعر 0) مملوك دائمًا. «نجمة الإتقان» مجانية لكن مقفلة حتى يكمل الطالب درسًا. تغيير اللون يغيّر لون الملف كله (الأفاتار والبطاقات). اللبس يُحفظ في تفضيلات الطالب (`pref.<id>.equipped`).

الإكسسوارات مرسومة بالكود (`AccessoryPainter`) على لوحة 150×150 نفسها التي رُسمت عليها الشخصية، فتتطابق مواضعها عند أي حجم بلا ملفات صور.

lib/features/wardrobe/wardrobe_items.dart — عناصر الخزانة

`lib/features/wardrobe/wardrobe_items.dart`

الأسطر 1–4

```dart
import '../../core/db/enums.dart';
import '../../core/strings/ar_progress.dart';

enum WardrobeKind { accessory, color }
```

ماذا يفعل؟ استيراد ألوان الشخصية والنصوص، وتعداد نوع العنصر (إكسسوار/لون).

لماذا بهذه الطريقة؟ الأسماء الظاهرة للطالب من `ArProgress` (ملف النصوص).

الأسطر 6–13

```dart
/// شرط فتح عنصر لا يُشترى بالنجوم وحدها.
enum Unlock {
  /// متاح بالنجوم فقط.
  none,

  /// يلزم إكمال درس واحد على الأقل.
  lessonCompleted,
}
```

ماذا يفعل؟ شروط الفتح: لا شرط، أو إكمال درس.

لماذا بهذه الطريقة؟ تعداد يتسع لشروط أخرى لاحقًا.

الأسطر 15–37

```dart
class WardrobeItem {
  const WardrobeItem({
    required this.id,
    required this.kind,
    required this.name,
    required this.price,
    this.unlock = Unlock.none,
    this.color,
  });

  final String id;
  final WardrobeKind kind;
  final String name;

  /// 0 = مجاني ويُعدّ مملوكًا دائمًا.
  final int price;
  final Unlock unlock;

  /// لون الشخصية الذي يعطيه عنصر اللون.
  final ProfileColor? color;

  bool get isFree => price == 0 && unlock == Unlock.none;
}
```

ماذا يفعل؟ `WardrobeItem`: المعرّف والنوع والاسم والسعر والشرط ولون الشخصية، و`isFree` (سعر 0 بلا شرط).

لماذا بهذه الطريقة؟ عنصر ثابت `const`: بيانات لا تتغيّر وقت التشغيل.

الأسطر 39–46

```dart
/// عناصر الخزانة. الأسعار بالنجوم؛ المجاني (0) متاح للجميع من البداية.
const wardrobeItems = [
  WardrobeItem(
    id: 'cap',
    kind: WardrobeKind.accessory,
    name: ArProgress.itemCap,
    price: 0,
  ),
```

ماذا يفعل؟ قائمة العناصر تبدأ بقبعة التخرّج (مجانية).

لماذا بهذه الطريقة؟ القبعة تعطي كل طالب شيئًا يلبسه من البداية.

الأسطر 47–58

```
    WardrobeItem(
      id: 'scarf',
      kind: WardrobeKind.accessory,
      name: ArProgress.itemScarf,
      price: 20,
    ),
    WardrobeItem(
      id: 'glasses',
      kind: WardrobeKind.accessory,
      name: ArProgress.itemGlasses,
      price: 30,
    ),
```

ماذا يفعل؟ الشال (20 نجمة) والنظارة (30).

لماذا بهذه الطريقة؟ أسعار صغيرة تُنال بجولة أو اثنتين.

الأسطر 59–108

```
  WardrobeItem(
    id: 'star',
    kind: WardrobeKind.accessory,
    name: ArProgress.itemStar,
    price: 0,
    unlock: Unlock.lessonCompleted,
  ),
  WardrobeItem(
    id: 'color.teal',
    kind: WardrobeKind.color,
    name: ArProgress.colorTeal,
    price: 0,
    color: ProfileColor.teal,
  ),
  WardrobeItem(
    id: 'color.coral',
    kind: WardrobeKind.color,
    name: ArProgress.colorCoral,
    price: 40,
    color: ProfileColor.coral,
  ),
  WardrobeItem(
    id: 'color.blue',
    kind: WardrobeKind.color,
    name: ArProgress.colorBlue,
    price: 40,
    color: ProfileColor.blue,
  ),
  WardrobeItem(
    id: 'color.purple',
    kind: WardrobeKind.color,
    name: ArProgress.colorPurple,
    price: 50,
    color: ProfileColor.purple,
  ),
  WardrobeItem(
    id: 'color.amber',
    kind: WardrobeKind.color,
    name: ArProgress.colorAmber,
    price: 50,
    color: ProfileColor.amber,
  ),
];

WardrobeItem? wardrobeItemById(String id) {
  for (final i in wardrobeItems) {
    if (i.id == id) return i;
  }
  return null;
}
```

ماذا يفعل؟ نجمة الإتقان (مقفلة بدرس) ثم الألوان: الأخضر مجاني، البرتقالي والسماوي 40، البنفسجي والعسلي 50، ودالة `wardrobeItemById`.

لماذا بهذه الطريقة؟ الأخضر مجاني ليستطيع الطالب الرجوع للون الأصلي.

lib/features/wardrobe/wardrobe_repository.dart — الشراء واللبس

lib/features/wardrobe/wardrobe_repository.dart

الأسطر 1–2

```dart
import '../../core/db/app_database.dart';
import 'wardrobe_items.dart';
```

ماذا يفعل؟ الاستيرادات.

لماذا بهذه الطريقة؟

الأسطر 4–13

```dart
enum PurchaseResult {
  purchased,
  notEnoughStars,
  locked,
  alreadyOwned,
  unknownItem,
}

/// حالة عنصر بالنسبة لطالب.
enum ItemState { equipped, owned, buyable, locked }
```

ماذا يفعل؟ نتائج الشراء وحالات العنصر.

لماذا بهذه الطريقة؟ تعدادات تجعل الشاشة تردّ بنص مناسب لكل نتيجة.

الأسطر 15–22

```dart
/// خزانة الشخصية: الرصيد والشراء واللبس. الرصيد = نجوم الطالب - ما أنفقه، فلا
/// يُخزَّن رقمٌ قد يخرج عن التزامن مع المحاولات.
class WardrobeRepository {
  WardrobeRepository(this._db, {DateTime Function()? now})
    : _now = now ?? DateTime.now;

  final AppDatabase _db;
  final DateTime Function() _now;
```

ماذا يفعل؟ `WardrobeRepository` والتوثيق: الرصيد يُحسب ولا يُخزَّن.

لماذا بهذه الطريقة؟ رقم مخزَّن قد يخرج عن التزامن عند تعديل المحاولات.

السطر 24

```dart
  static String _equippedKey(String profileId) => 'pref.$profileId.equipped';
```

ماذا يفعل؟ مفتاح تفضيل المرتدى.

لماذا بهذه الطريقة؟ يعيد استعمال جدول الإعدادات (لا ترحيل).

الأسطر 26–31

```
  /// معرّفات العناصر المملوكة (المجانية تُحتسب مملوكة دائمًا).
  static Set<String> ownedIds(Iterable<OwnedItem> purchased) => {
    for (final i in wardrobeItems)
      if (i.isFree) i.id,
    for (final p in purchased) p.itemId,
  };
```

ماذا يفعل؟ `ownedIds`: المجاني + ما اشتُري.

لماذا بهذه الطريقة؟

الأسطر 33–36

```
  static int balance({
    required int stars,
    required Iterable<OwnedItem> owned,
  }) => stars - owned.fold<int>(0, (s, o) => s + o.cost);
```

ماذا يفعل؟ `balance`: النجوم − مجموع التكاليف المسجّلة وقت الشراء.

لماذا بهذه الطريقة؟ التكلفة تُحفظ وقت الشراء فلا يتغيّر رصيد الطالب لو عُدّلت الأسعار.

الأسطر 38–61

```
  /// حالة العنصر: مرتدى / مملوك / قابل للشراء / مقفل بشرطه.
  static ItemState stateOf(
    WardrobeItem item, {
    required Set<String> owned,
    required Set<String> equipped,
    required bool lessonCompleted,
    required String currentColorItemId,
  }) {
    final isOwned =
        owned.contains(item.id) ||
        (item.unlock == Unlock.lessonCompleted &&
            lessonCompleted &&
            item.price == 0);
    if (isOwned) {
      final on = item.kind == WardrobeKind.color
          ? item.id == currentColorItemId
          : equipped.contains(item.id);
      return on ? ItemState.equipped : ItemState.owned;
    }
    if (item.unlock == Unlock.lessonCompleted && !lessonCompleted) {
      return ItemState.locked;
    }
    return ItemState.buyable;
  }
```

ماذا يفعل؟ `stateOf`: مرتدى / مملوك / قابل للشراء / مقفل. اللون «مرتدى» إن كان لون الملف الحالي.

لماذا بهذه الطريقة؟ دالة نقية تُختبر بلا قاعدة (حالتان).

الأسطر 63–74

```
  Stream<Set<String>> watchEquipped(String profileId) {
    final key = _equippedKey(profileId);
    return _db.settingsDao.watchByPrefix(key).map((rows) {
      final raw = rows[key];
      return raw == null || raw.isEmpty ? <String>{} : raw.split(',').toSet();
    });
  }

  Future<Set<String>> _equipped(String profileId) async {
    final raw = await _db.settingsDao.read(_equippedKey(profileId));
    return raw == null || raw.isEmpty ? <String>{} : raw.split(',').toSet();
  }
```

ماذا يفعل؟ `watchEquipped` و`_equipped`: قراءة المرتدى كقائمة مفصولة بفواصل.

لماذا بهذه الطريقة؟

الأسطر 76–81

```
  Future<bool> _lessonCompleted(String profileId) async {
    final rows = await (_db.select(
      _db.lessonProgress,
    )..where((r) => r.profileId.equals(profileId))).get();
    return rows.any((r) => r.status == LessonStatus.completed);
  }
```

ماذا يفعل؟ `_lessonCompleted`: هل أكمل الطالب درسًا؟

لماذا بهذه الطريقة؟ شرط نجمة الإتقان.

الأسطر 83–106

```
  /// يشتري [itemId] إن كان ممكنًا. كل الفحوص والكتابة في معاملة واحدة.
  Future<PurchaseResult> buy(String profileId, String itemId) {
    final item = wardrobeItemById(itemId);
    if (item == null) return Future.value(PurchaseResult.unknownItem);
    return _db.transaction(() async {
      final owned = await _db.wardrobeDao.getOwned(profileId);
      if (ownedIds(owned).contains(itemId)) return PurchaseResult.alreadyOwned;
      if (item.unlock == Unlock.lessonCompleted &&
          !await _lessonCompleted(profileId)) {
        return PurchaseResult.locked;
      }
      final stars = await _db.activityDao.totalStars(profileId);
      if (balance(stars: stars, owned: owned) < item.price) {
        return PurchaseResult.notEnoughStars;
      }
      final ok = await _db.wardrobeDao.insertPurchase(
        profileId: profileId,
        itemId: itemId,
        cost: item.price,
        now: _now(),
      );
      return ok ? PurchaseResult.purchased : PurchaseResult.alreadyOwned;
    });
  }
```

ماذا يفعل؟ `buy` في معاملة: مملوك؟ مقفل؟ الرصيد؟ ثم الإدخال. أي نتيجة إدخال مكرّر = «مملوك».

لماذا بهذه الطريقة؟ المعاملة + المفتاح المركّب = لا خصم مزدوج (اختبار بنقرتين متزامنتين).

الأسطر 108–122

```
  /// يلبس/يخلع إكسسوارًا مملوكًا. false إن لم يكن مملوكًا أو كان لونًا.
  Future<bool> toggleAccessory(String profileId, String itemId) async {
    final item = wardrobeItemById(itemId);
    if (item == null || item.kind != WardrobeKind.accessory) return false;
    final owned = ownedIds(await _db.wardrobeDao.getOwned(profileId));
    final unlockedFree =
        item.unlock == Unlock.lessonCompleted &&
        item.price == 0 &&
        await _lessonCompleted(profileId);
    if (!owned.contains(itemId) && !unlockedFree) return false;
    final set = await _equipped(profileId);
    if (!set.add(itemId)) set.remove(itemId);
    await _db.settingsDao.write(_equippedKey(profileId), set.join(','));
    return true;
  }
```

ماذا يفعل؟ `toggleAccessory`: يلبس/يخلع إكسسوارًا مملوكًا أو مفتوحًا مجانًا.

لماذا بهذه الطريقة؟ غير المملوك لا يُلبس.

الأسطر 124–133

```
  /// يغيّر لون الشخصية إلى لون عنصر مملوك.
  Future<bool> useColor(String profileId, String itemId) async {
    final item = wardrobeItemById(itemId);
    if (item == null || item.kind != WardrobeKind.color) return false;
    final owned = ownedIds(await _db.wardrobeDao.getOwned(profileId));
    if (!owned.contains(itemId)) return false;
    await _db.profilesDao.setColor(profileId, item.color!);
    return true;
  }
}
```

ماذا يفعل؟ `useColor`: يغيّر لون الملف إلى لون عنصر مملوك.

لماذا بهذه الطريقة؟

lib/features/wardrobe/wardrobe_providers.dart — المزوّدات

lib/features/wardrobe/wardrobe_providers.dart

الأسطر 1–6

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../core/db/app_database.dart';
import '../../core/db/database_provider.dart';
import '../profiles/active_profile_provider.dart';
import 'wardrobe_repository.dart';
```

ماذا يفعل؟ الاستيرادات.

لماذا بهذه الطريقة؟

الأسطر 8–13

```
final wardrobeRepositoryProvider = Provider<WardrobeRepository>((ref) {
  return WardrobeRepository(
    ref.watch(appDatabaseProvider),
    now: ref.watch(clockProvider),
  );
});
```

ماذا يفعل؟ مزوّد المستودع بالساعة المحقونة.

لماذا بهذه الطريقة؟

الأسطر 15–25

```
final ownedItemsProvider = StreamProvider.autoDispose
    .family<List<OwnedItem>, String>(
      (ref, profileId) =>
          ref.watch(appDatabaseProvider).wardrobeDao.watchOwned(profileId),
    );

final equippedItemsProvider = StreamProvider.autoDispose
    .family<Set<String>, String>(
      (ref, profileId) =>
          ref.watch(wardrobeRepositoryProvider).watchEquipped(profileId),
    );
```

ماذا يفعل؟ بثّ المشتريات والمرتدى لطالب.

لماذا بهذه الطريقة؟ تتحدّث الشاشة فور الشراء أو اللبس.

الأسطر 27–32

```
/// الإكسسوارات التي يرتديها الملف النشط (فارغة قبل التحميل).
final activeEquippedProvider = Provider.autoDispose<Set<String>>((ref) {
  final id = ref.watch(activeProfileIdProvider).value;
  if (id == null) return const {};
  return ref.watch(equippedItemsProvider(id)).value ?? const {};
});
```

ماذا يفعل؟ `activeEquippedProvider`: مرتدى الملف النشط (يستعمله «حسابي» والرئيسية الصغيرة).

لماذا بهذه الطريقة؟

lib/core/widgets/mascot_items.dart — رسم الإكسسوارات

lib/core/widgets/mascot_items.dart

الأسطر 1–7

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

import '../db/enums.dart';
import '../theme/app_colors.dart';
import 'mascot.dart';
```

ماذا يفعل؟ الاستيرادات: `dart:math` (للنجمة) وMaterial والتعداد والتميمة.

لماذا بهذه الطريقة؟ الإكسسوارات رسم بالكود لا صور، فتتلوّن وتتقيّس مع التميمة.

الأسطر 9–24

```dart
/// شخصية رِزمة مع إكسسواراتها المرتداة. الإكسسوارات مرسومة بالكود على لوحة
/// 150×150 نفسها التي رُسمت عليها الشخصية، فتتطابق مواضعها عند أي حجم.
/// المواضع مضبوطة على تميمة التصميم الجديد (الرأس عند y≈24 والعينان عند y≈77).
class MascotWithItems extends StatelessWidget {
  const MascotWithItems({
    super.key,
    required this.size,
    this.color = ProfileColor.teal,
    this.items = const {},
  });

  final double size;
  final ProfileColor color;

  /// معرّفات الإكسسوارات: `cap` `scarf` `glasses` `star`.
  final Set<String> items;
```

ماذا يفعل؟ `MascotWithItems`: الحجم واللون ومجموعة الإكسسوارات الملبوسة.

لماذا بهذه الطريقة؟ ودجت واحد يُستعمل في الحساب والخزانة.

الأسطر 26–43

```dart
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: size,
      height: size,
      child: Stack(
        clipBehavior: Clip.none,
        children: [
          Mascot(size: size, color: color),
          if (items.isNotEmpty)
            Positioned.fill(
              child: CustomPaint(painter: AccessoryPainter(items)),
            ),
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ مكدّس: الشخصية ثم طبقة رسم فوقها بنفس الحجم (بلا قص).

لماذا بهذه الطريقة؟ القبعة تتجاوز رأس اللوحة قليلًا فلا نقصّ.

الأسطر 45–63

```dart
class AccessoryPainter extends CustomPainter {
  AccessoryPainter(this.items);

  final Set<String> items;

  static const _ink = AppColors.ink;
  static const _amber = Color(0xFFF2B650);
  static const _red = Color(0xFFB94F2A);

  @override
  void paint(Canvas canvas, Size size) {
    canvas.save();
    canvas.scale(size.width / 150, size.height / 150);
    if (items.contains('scarf')) _scarf(canvas);
    if (items.contains('glasses')) _glasses(canvas);
    if (items.contains('star')) _star(canvas);
    if (items.contains('cap')) _cap(canvas);
    canvas.restore();
  }
```

ماذا يفعل؟ `AccessoryPainter`: يقيس اللوحة من 150 إلى الحجم الفعلي ويرسم بالترتيب: شال، نظارة، نجمة، قبعة.

لماذا بهذه الطريقة؟ قياس واحد = تتطابق المواضع في كل حجم.

الأسطر 96–65

```
  void _cap(Canvas canvas) {
    final fill = Paint()..color = _ink;
    // القاعدة على الرأس ثم اللوح المعيّن فوقها.
    canvas.drawRRect(
      RRect.fromRectAndRadius(
        const Rect.fromLTWH(48, 18, 44, 16),
        const Radius.circular(5),
      ),
      fill,
    );
    final board = Path()
      ..moveTo(70, 6)
      ..lineTo(112, 19)
      ..lineTo(70, 32)
      ..lineTo(28, 19)
      ..close();
    canvas.drawPath(board, fill);
    canvas.drawPath(
      board,
      Paint()
        ..style = PaintingStyle.stroke
        ..strokeWidth = 1.5
        ..color = const Color(0xFF143C3B),
    );
    // الشرّابة.
    final tassel = Paint()
      ..color = _amber
      ..strokeWidth = 2.5
      ..strokeCap = StrokeCap.round;
    canvas.drawLine(const Offset(104, 21), const Offset(104, 38), tassel);
    canvas.drawCircle(const Offset(104, 40), 3.5, Paint()..color = _amber);
  }
```

ماذا يفعل؟ القبعة: قاعدة على رأس التميمة الجديدة (y=18) ولوح معيّن وشرّابة عنبرية.

لماذا بهذه الطريقة؟ تغيّر: المواضع أُعيد ضبطها لأن تميمة Figma الجديدة أعلى رأسًا من القديمة (القبعة نزلت 8 نقاط).

الأسطر 98–117

```
  void _scarf(Canvas canvas) {
    final band = RRect.fromRectAndRadius(
      const Rect.fromLTWH(27, 106, 96, 15),
      const Radius.circular(7),
    );
    canvas.drawRRect(band, Paint()..color = _red);
    // ذيل الشال المتدلّي.
    canvas.drawRRect(
      RRect.fromRectAndRadius(
        const Rect.fromLTWH(94, 114, 16, 24),
        const Radius.circular(5),
      ),
      Paint()..color = const Color(0xFF9A3F20),
    );
    // رقعات مطرّزة.
    final dot = Paint()..color = _amber;
    for (var x = 38.0; x < 120; x += 14) {
      canvas.drawCircle(Offset(x, 113.5), 2.4, dot);
    }
  }
```

ماذا يفعل؟ الشال: شريط عند y=106 حول أسفل الجسم وذيل متدلٍّ ونقاط مطرّزة.

لماذا بهذه الطريقة؟ موضع الشال على «رقبة» الدفتر في التميمة الجديدة.

الأسطر 119–127

```
  void _glasses(Canvas canvas) {
    final stroke = Paint()
      ..style = PaintingStyle.stroke
      ..strokeWidth = 3
      ..color = _ink;
    canvas.drawCircle(const Offset(63, 77), 14, stroke);
    canvas.drawCircle(const Offset(92, 77), 14, stroke);
    canvas.drawLine(const Offset(77, 76), const Offset(78, 76), stroke);
  }
```

ماذا يفعل؟ النظارة: دائرتان نصف قطرهما 14 على العينين (63,77) و(92,77) وجسر صغير.

لماذا بهذه الطريقة؟ إحداثيات العينين مأخوذة من SVG التميمة الجديدة بعد تحويلها إلى لوحة 150×150.

الأسطر 129–142

```
  void _star(Canvas canvas) {
    const c = Offset(42, 50);
    const outer = 10.0;
    const inner = 4.4;
    final path = Path();
    for (var i = 0; i < 10; i++) {
      final r = i.isEven ? outer : inner;
      final a = -math.pi / 2 + i * math.pi / 5;
      final p = Offset(c.dx + r * math.cos(a), c.dy + r * math.sin(a));
      i == 0 ? path.moveTo(p.dx, p.dy) : path.lineTo(p.dx, p.dy);
    }
    path.close();
    canvas.drawPath(path, Paint()..color = _amber);
  }
```

ماذا يفعل؟ النجمة عند (42,50): مضلّع بعشر رؤوس بالجيب والجيب التمام.

لماذا بهذه الطريقة؟ في الزاوية العليا بعيدًا عن الورقتين فوق الدفتر.

الأسطر 144–149

```
  @override
  bool shouldRepaint(AccessoryPainter old) => !_same(old.items, items);

  static bool _same(Set<String> a, Set<String> b) =>
      a.length == b.length && a.containsAll(b);
}
```

ماذا يفعل؟ `shouldRepaint` بمقارنة المجموعتين.

لماذا بهذه الطريقة؟ يُعاد الرسم فقط حين تتغيّر الإكسسوارات.

الشاشة 24 — lib/features/wardrobe/wardrobe_screen.dart

`lib/features/wardrobe/wardrobe_screen.dart`

الأسطر 1–20

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../../app/routes.dart';
import '../../core/db/enums.dart';
import '../../core/strings/ar_progress.dart';
import '../../core/theme/app_colors.dart';
import '../../core/widgets/ledge.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/utils/haptics.dart';
import '../../core/widgets/app_icon.dart';
import '../../core/widgets/mascot_items.dart';
import '../../core/widgets/screen_top_bar.dart';
import '../home/home_providers.dart';
import '../learning/widgets/round_panels.dart';
import '../profiles/active_profile_provider.dart';
import 'wardrobe_items.dart';
import 'wardrobe_providers.dart';
import 'wardrobe_repository.dart';
```

ماذا يفعل؟ الاستيرادات.

لماذا بهذه الطريقة؟

السطر 22

```dart
enum _Filter { all, colors, accessories }
```

ماذا يفعل؟ مرشّحات: الكل والألوان والإكسسوارات.

لماذا بهذه الطريقة؟

الأسطر 24–34

```dart
/// الشاشة 24 «خزانة رزمة»: يزيّن الطالب شخصيته بنجومه. النجوم تفتح الزينة
/// فقط، أما الدروس فمفتوحة دائمًا.
class WardrobeScreen extends ConsumerStatefulWidget {
  const WardrobeScreen({super.key});

  @override
  ConsumerState<WardrobeScreen> createState() => _WardrobeScreenState();
}

class _WardrobeScreenState extends ConsumerState<WardrobeScreen> {
  _Filter _filter = _Filter.all;
```

ماذا يفعل؟ `WardrobeScreen` بحالة المرشّح.

لماذا بهذه الطريقة؟

الأسطر 36–40

```
36    void _say(String text) {
37      ScaffoldMessenger.of(context)
38        ..hideCurrentSnackBar()
39        ..showSnackBar(SnackBar(content: Text(text)));
40    }
```

ماذا يفعل؟ `_say`: رسالة سفلية بعد إخفاء السابقة.

لماذا بهذه الطريقة؟

الأسطر 42–57

```
42    Future<void> _onTap(
43      WardrobeItem item,
44      ItemState state,
45      String profileId,
46      int balance,
47    ) async {
48      final repo = ref.read(wardrobeRepositoryProvider);
49      switch (state) {
50        case ItemState.equipped:
51        case ItemState.owned:
52          Haptics.tap();
53          if (item.kind == WardrobeKind.accessory) {
54            await repo.toggleAccessory(profileId, item.id);
55          } else {
56            await repo.useColor(profileId, item.id);
57          }
```

ماذا يفعل؟ `_onTap`: المملوك يُلبس/يُخلع أو يُستعمل لونًا.

لماذا بهذه الطريقة؟ نقرة واحدة تبدّل.

الأسطر 58–64

```
58        case ItemState.locked:
59          _say(ArProgress.lockedLesson);
60        case ItemState.buyable:
61          if (balance < item.price) {
62            _say(ArProgress.notEnoughStars);
63            return;
64          }
```

ماذا يفعل؟ المقفل يخبر بالسبب، والشراء بنجوم لا تكفي يخبر بذلك.

لماذا بهذه الطريقة؟

الأسطر 92–65

```
65          final ok = await showDialog<bool>(
66            context: context,
67            builder: (context) => AlertDialog(
68              title: const Text(ArProgress.confirmBuyTitle),
69              content: Text(ArProgress.confirmBuyBody(item.name, item.price)),
70              actions: [
71                TextButton(
72                  onPressed: () => Navigator.of(context).pop(false),
73                  child: const Text(ArProgress.cancel),
74                ),
75                TextButton(
76                  onPressed: () => Navigator.of(context).pop(true),
77                  child: const Text(ArProgress.buy),
78                ),
79              ],
80            ),
81          );
82          if (ok != true || !mounted) return;
83          final result = await repo.buy(profileId, item.id);
84          if (!mounted) return;
85          Haptics.heavy();
86          _say(
87            result == PurchaseResult.purchased
88                ? ArProgress.purchased(item.name)
89                : ArProgress.notEnoughStars,
90          );
91        }
92      }
```

ماذا يفعل؟ الشراء: حوار تأكيد ثم الشراء ثم رسالة النتيجة.

لماذا بهذه الطريقة؟ الأطفال قد ينقرون عرضًا؛ التأكيد يحمي نجومهم.

الأسطر 94–121

```
94    @override
95    Widget build(BuildContext context) {
96      final profile = ref.watch(activeProfileProvider).value;
97      if (profile == null) return const Scaffold(body: SizedBox.shrink());
98
99      final owned = ref.watch(ownedItemsProvider(profile.id)).value ?? const [];
100     final equipped =
101         ref.watch(equippedItemsProvider(profile.id)).value ?? const <String>{};
102     final attempts = ref.watch(attemptsProvider(profile.id)).value ?? const [];
103     final progress =
104         ref.watch(lessonProgressProvider(profile.id)).value ?? const [];
105
106     final stars = attempts.fold<int>(0, (s, a) => s + a.starsEarned);
107     final balance = WardrobeRepository.balance(stars: stars, owned: owned);
108     final ownedSet = WardrobeRepository.ownedIds(owned);
109     final lessonDone = progress.any((p) => p.status == LessonStatus.completed);
110     final currentColorId = wardrobeItems
111         .firstWhere((i) => i.color == profile.color)
112         .id;
113
114     ItemState stateOf(WardrobeItem i) => WardrobeRepository.stateOf(
115       i,
116       owned: ownedSet,
117       equipped: equipped,
118       lessonCompleted: lessonDone,
119       currentColorItemId: currentColorId,
120     );
```

ماذا يفعل؟ `build` : بيانات الملف والمملوك والمرتدى والمحاولات والتقدّم، والرصيد والحالة لكل عنصر.

لماذا بهذه الطريقة؟

الأسطر 122–136

```
122     final shown = [
123       for (final i in wardrobeItems)
124         if (_filter == _Filter.all ||
125             (_filter == _Filter.colors && i.kind == WardrobeKind.color) ||
126             (_filter == _Filter.accessories &&
127                 i.kind == WardrobeKind.accessory))
128           i,
129     ];
130     final wornNames = [
131       for (final i in wardrobeItems)
132         if (i.kind == WardrobeKind.accessory &&
133             equipped.contains(i.id) &&
134             stateOf(i) == ItemState.equipped)
135           i.name,
136     ];
```

ماذا يفعل؟ العناصر المعروضة بحسب المرشّح، وأسماء المرتدى للتسمية تحت الصورة.

لماذا بهذه الطريقة؟

الأسطر 166–138

```
    return Scaffold(
      body: SafeArea(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            ScreenTopBar(
              title: ArProgress.wardrobeTitle,
              subtitle: ArProgress.wardrobeKicker(profile.name),
              onBack: () => context.canPop()
                  ? context.pop()
                  : context.go(AppRoutes.account),
              end: StarsChip(count: balance),
            ),
            Expanded(
              child: SingleChildScrollView(
                padding: const EdgeInsetsDirectional.fromSTEB(20, 12, 20, 24),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  spacing: 14,
                  children: [
                    _Hero(
                      color: profile.color,
                      worn: equipped,
                      caption: wornNames.join(' · '),
                    ),
                    _Filters(
                      current: _filter,
                      onChanged: (f) => setState(() => _filter = f),
                    ),
```

ماذا يفعل؟ الهيكل: العنوان والرصيد وبطاقة الشخصية والمرشّحات.

لماذا بهذه الطريقة؟

الأسطر 167–195

```
                  LayoutBuilder(
                    builder: (context, c) {
                      const gap = 10.0;
                      final width = (c.maxWidth - 2 * gap) / 3;
                      return Wrap(
                        spacing: gap,
                        runSpacing: gap,
                        children: [
                          for (final item in shown)
                            SizedBox(
                              width: width,
                              child: _ItemCard(
                                item: item,
                                state: stateOf(item),
                                profileColor: profile.color,
                                onTap: () => _onTap(
                                  item,
                                  stateOf(item),
                                  profile.id,
                                  balance,
                                ),
                              ),
                            ),
                        ],
                      );
                    },
                  ),
                  LedgeBox(
                    radius: 16,
```

ماذا يفعل؟ شبكة ثلاثية الأعمدة بـ `LayoutBuilder` و`Wrap`، وكل عنصر بطاقة `LedgeBox`.

لماذا بهذه الطريقة؟ عرض العمود يُحسب من العرض المتاح فتعمل على كل الشاشات. البطاقات بحافة سفلية من التصميم الجديد.

الأسطر 196–225

```
                      child: Padding(
                        padding: const EdgeInsets.all(14),
                        child: Row(
                          spacing: 10,
                          children: [
                            const AppIcon(
                              'shield',
                              size: 22,
                              color: AppColors.teal,
                            ),
                            Expanded(
                              child: Text(
                                ArProgress.wardrobeNote,
                                style: AppTextStyles.label14,
                              ),
                            ),
                          ],
                        ),
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ ملاحظة «النجوم تفتح الزينة فقط».

لماذا بهذه الطريقة؟ الصدق: لا شيء من التعلّم يُحجب.

الأسطر 227–258

```dart
class _Hero extends StatelessWidget {
  const _Hero({required this.color, required this.worn, required this.caption});

  final ProfileColor color;
  final Set<String> worn;
  final String caption;

  @override
  Widget build(BuildContext context) {
    return DecoratedBox(
      decoration: BoxDecoration(
        color: AppColors.tealTint,
        borderRadius: BorderRadius.circular(24),
      ),
      child: Padding(
        padding: const EdgeInsets.fromLTRB(16, 20, 16, 14),
        child: Column(
          spacing: 8,
          children: [
            MascotWithItems(size: 150, color: color, items: worn),
            if (caption.isNotEmpty)
              Text(
                caption,
                textAlign: TextAlign.center,
                style: AppTextStyles.body15.copyWith(color: AppColors.tealDark),
              ),
          ],
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ `_Hero` : الشخصية الكبيرة بما ترتديه وأسماؤه.

لماذا بهذه الطريقة؟

الأسطر 260–307

```
260 class _Filters extends StatelessWidget {
261   const _Filters({required this.current, required this.onChanged});
262
263   final _Filter current;
264   final ValueChanged<_Filter> onChanged;
265
266   @override
267   Widget build(BuildContext context) {
268     const labels = {
269       _Filter.all: ArProgress.filterAll,
270       _Filter.colors: ArProgress.filterColors,
271       _Filter.accessories: ArProgress.filterAccessories,
272     };
273     return Wrap(
274       spacing: 10,
275       runSpacing: 8,
276       children: [
277         for (final f in _Filter.values)
278           Semantics(
279             button: true,
280             selected: f == current,
281             label: labels[f],
282             excludeSemantics: true,
283             child: Material(
284               color: f == current ? AppColors.teal : AppColors.mute,
285               borderRadius: BorderRadius.circular(999),
286               child: InkWell(
287                 borderRadius: BorderRadius.circular(999),
288                 onTap: () => onChanged(f),
289                 child: Padding(
290                   padding: const EdgeInsets.symmetric(
291                     horizontal: 20,
292                     vertical: 8,
293                   ),
294                   child: Text(
295                     labels[f]!,
296                     style: AppTextStyles.body15.copyWith(
297                       color: f == current ? AppColors.card : AppColors.ink,
298                     ),
299                   ),
300                 ),
301               ),
302             ),
303           ),
304       ],
305     );
306   }
307 }
```

ماذا يفعل؟ `_Filters` : ثلاث حبّات.

لماذا بهذه الطريقة؟

الأسطر 309–357

```
class _ItemCard extends StatelessWidget {
  const _ItemCard({
    required this.item,
    required this.state,
    required this.profileColor,
    required this.onTap,
  });

  final WardrobeItem item;
  final ItemState state;
  final ProfileColor profileColor;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final equipped = state == ItemState.equipped;
    final locked = state == ItemState.locked;

    final Widget chip = switch (state) {
      ItemState.equipped => _Chip(
        label: item.kind == WardrobeKind.color
            ? ArProgress.itemInUse
            : ArProgress.itemEquipped,
        icon: 'check',
        fg: AppColors.teal,
        bg: AppColors.tealTint,
      ),
      ItemState.owned => _Chip(
        label: item.kind == WardrobeKind.color
            ? ArProgress.itemUseColor
            : ArProgress.itemOwned,
        fg: AppColors.soft,
        bg: AppColors.mute,
      ),
      ItemState.buyable => _Chip(
        label: ArProgress.itemPrice(item.price),
        icon: 'starf',
        fg: AppColors.amberDark,
        bg: AppColors.amberTint,
        iconAfter: true,
      ),
      ItemState.locked => const _Chip(
        label: ArProgress.lockedLesson,
        icon: 'lock',
        fg: AppColors.soft,
        bg: AppColors.mute,
      ),
    };
```

ماذا يفعل؟ `_ItemCard` وحالة الحبّة تحت العنصر (مرتدى/استخدم، مملوك، سعر، مقفل).

لماذا بهذه الطريقة؟

الأسطر 358–414

```
    return Semantics(
      button: true,
      label: item.name,
      child: Material(
        color: AppColors.card,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(20),
          side: BorderSide(
            color: equipped ? AppColors.teal : AppColors.line,
            width: equipped ? 2 : 1,
          ),
        ),
        clipBehavior: Clip.antiAlias,
        child: InkWell(
          onTap: onTap,
          child: Padding(
            padding: const EdgeInsets.fromLTRB(6, 12, 6, 12),
            child: Opacity(
              opacity: locked ? 0.6 : 1,
              child: Column(
                children: [
                  MascotWithItems(
                    size: 56,
                    color: item.kind == WardrobeKind.color
                        ? item.color!
                        : profileColor,
                    items: item.kind == WardrobeKind.accessory
                        ? {item.id}
                        : const {},
                  ),
                  const SizedBox(height: 6),
                  SizedBox(
                    height: 40,
                    child: Center(
                      child: Text(
                        item.name,
                        textAlign: TextAlign.center,
                        maxLines: 2,
                        overflow: TextOverflow.ellipsis,
                        style: AppTextStyles.captionBold13.copyWith(
                          color: AppColors.ink,
                          height: 1.25,
                        ),
                      ),
                    ),
                  ),
                  const SizedBox(height: 4),
                  chip,
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ البطاقة: معاينة الشخصية بالعنصر (أو باللون) واسمه، والقفل بشفافية 0.6.

لماذا بهذه الطريقة؟ ترى الطالب ما سيحصل عليه قبل أن يشتريه.

الأسطر 416–461

```dart
class _Chip extends StatelessWidget {
  const _Chip({
    required this.label,
    required this.fg,
    required this.bg,
    this.icon,
    this.iconAfter = false,
  });

  final String label;
  final String? icon;
  final Color fg;
  final Color bg;
  final bool iconAfter;

  @override
  Widget build(BuildContext context) {
    final text = Flexible(
      child: Text(
        label,
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
        style: AppTextStyles.chip13.copyWith(color: fg, fontSize: 12),
      ),
    );
    final glyph = icon == null ? null : AppIcon(icon!, size: 14, color: fg);
    return DecoratedBox(
      decoration: BoxDecoration(
        color: bg,
        borderRadius: BorderRadius.circular(999),
      ),
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
        child: Row(
          mainAxisSize: MainAxisSize.min,
          spacing: 4,
          children: [
            if (glyph != null && !iconAfter) glyph,
            text,
            if (glyph != null && iconAfter) glyph,
          ],
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ `_Chip` : حبّة بأيقونة قبل النص أو بعده.

لماذا بهذه الطريقة؟ النجمة بعد الرقم عند السعر.